

GenAI Developer Interview Preparation

130 Interview Questions & Answers
50 Basic + 50 Intermediate + 30 Advanced

Includes practical interview scenarios covering RAG, tokens, embeddings, vector databases, chunking, hybrid search, reranking, agents, tools, security, evaluation and production architecture.

Designed as a one-week interview revision guide

How to Use This Guide

Do not try to memorize every sentence. First understand the flow: User Question -> Prompt -> Tokens -> Embedding -> Retrieval/RAG -> LLM -> Tools/Agents -> Response -> Evaluation/Monitoring. Practice answering each question aloud in your own words.

Important: When discussing project experience, describe work truthfully as a hands-on project or proof of concept if it was not a production implementation.

Part 1 - 50 Basic Questions

1. What is Generative AI?

Generative AI is AI that can create new content such as text, code, images, audio, and video based on patterns learned during training.

2. What is an LLM?

A Large Language Model is a neural network trained on huge amounts of text to understand and generate language by predicting tokens. Examples include GPT, Claude, Gemini, and Llama.

3. What is GPT?

GPT means Generative Pre-trained Transformer. Generative means it creates content, pre-trained means it learns before application use, and Transformer is the underlying neural-network architecture.

4. What is a token?

A token is a small unit of text processed by an LLM. It may be a word, part of a word, punctuation, or another text fragment.

5. Why are tokens important?

Tokens affect context-window usage, API cost, latency, and how much information we can send to or receive from a model.

6. What is tokenization?

Tokenization converts text into tokens that can be processed numerically by the model.

7. What is a prompt?

A prompt is the instruction and context we provide to an LLM to guide its response.

8. What is prompt engineering?

Prompt engineering is designing prompts so the model produces more accurate, useful, structured, and predictable responses.

9. What is zero-shot prompting?

Asking the model to perform a task without providing examples.

10. What is few-shot prompting?

Providing a few examples of desired inputs and outputs before asking the model to handle a new input.

11. What is a system prompt?

A system prompt provides high-level instructions defining the model's role, behavior, constraints, and response rules.

12. What is temperature?

Temperature controls randomness in generation. Lower values generally make output more deterministic; higher values make it more varied.

13. What is hallucination?

Hallucination occurs when an LLM produces information that sounds plausible but is unsupported or incorrect.

14. How can hallucination be reduced?

Use grounding/RAG, strong prompts, authoritative sources, constrained output, tool calls, validation, and evaluation.

15. What is an embedding?

An embedding is a numerical vector representation of data such as text, designed so semantically related content is located relatively close in vector space.

16. Why do we need embeddings?

They allow applications to compare semantic meaning mathematically and perform semantic search.

17. What is a vector?

In GenAI, a vector is an array of numbers representing features or meaning of content.

18. What is a vector database?

A vector database stores embeddings and efficiently retrieves vectors similar to a query vector.

19. Give examples of vector databases.

Pinecone, Weaviate, Milvus and Qdrant are examples. PostgreSQL with pgvector and search platforms such as Elasticsearch can also support vector search.

20. What is semantic search?

Semantic search finds information based on meaning rather than only exact keyword matches.

21. What is cosine similarity?

It measures the similarity between vectors based on the angle between them. It is commonly used in semantic retrieval.

22. What is RAG?

RAG means Retrieval-Augmented Generation. It retrieves relevant information from an external knowledge source and supplies it to the LLM as context before generation.

23. Why use RAG?

It allows an LLM application to answer using private, current, or domain-specific information without retraining the model.

24. Explain RAG simply.

Search first, then generate. We retrieve relevant information and give it to the LLM along with the user's question.

25. What is chunking?

Chunking means dividing large documents into smaller pieces before generating embeddings.

26. Why is chunking required?

Smaller chunks allow retrieval of specific sections relevant to a question and avoid sending entire large documents.

27. What is chunk overlap?

It means sharing some text between adjacent chunks so important information is not lost at chunk boundaries.

28. What is a retriever?

A retriever searches a knowledge source and returns the most relevant documents or chunks for a query.

29. What is Top-K?

Top-K means retrieving the K highest-ranked results. Top-K = 5 means return the five most relevant chunks.

30. What is a context window?

The context window is the maximum amount of tokenized information a model can consider in one interaction.

31. What is fine-tuning?

Fine-tuning further trains an existing model on task-specific examples to modify its behavior for a particular use case.

32. RAG vs fine-tuning?

RAG primarily provides external knowledge at runtime. Fine-tuning changes model behavior through additional training. They solve different problems and can be combined.

33. What is an AI agent?

An AI agent is a system where an AI model can reason about a goal, choose actions or tools, observe results, and continue until it produces an answer or completes a task.

34. What is a tool in Agentic AI?

A tool is an external capability available to an agent, such as searching a database, calling an API, reading a file, or creating a ticket.

35. What is function calling?

Function calling allows the model to produce structured arguments indicating that the application should invoke a predefined function or tool.

36. What is an AI workflow?

A workflow defines controlled steps through which data, models, tools, validations, and business logic execute.

37. What is LangChain?

LangChain is a framework for building LLM applications involving models, prompts, retrieval, tools, agents, and integrations.

38. What is LangGraph?

LangGraph is a framework for building stateful, graph-based agent workflows with nodes, transitions, state and control over execution.

39. What is a transformer?

A Transformer is a neural-network architecture that uses attention mechanisms to model relationships among tokens and supports highly parallel training.

40. What is attention?

Attention lets a transformer weigh which parts of the input are most relevant when processing or generating information.

41. What is self-attention?

Self-attention calculates relationships between tokens within the same sequence to build context-aware representations.

42. What is inference?

Inference is using a trained model to generate predictions or responses for new input.

43. Training vs inference?

Training adjusts model parameters using data. Inference uses the already-trained model to answer requests.

44. What is grounding?

Grounding means basing an AI response on supplied, trusted information rather than relying only on model knowledge.

45. What is a multimodal model?

A multimodal model can process or generate multiple types of information such as text, images, audio, and video.

46. What is an API in GenAI?

An API lets our application programmatically send requests to an AI model or service and receive responses.

47. What is a model parameter?

Model parameters are learned numerical values, such as neural-network weights, adjusted during training.

48. What is structured output?

Structured output makes the model return information in a defined format or schema, commonly JSON, rather than unrestricted text.

49. What is a guardrail?

A guardrail is a control used to restrict or validate model inputs, outputs, or actions for safety, security, compliance, or business requirements.

50. What does a GenAI Developer do?

A GenAI developer integrates models with applications, prompts, RAG, vector databases, APIs, tools, agents, evaluation, security, observability, and business systems.

Part 2 - 50 Intermediate Questions

51. Explain the complete RAG pipeline.

Load documents -> clean/process -> chunk -> create embeddings -> store vectors -> embed user query -> similarity search -> retrieve relevant chunks -> optionally rerank/filter -> construct grounded prompt -> LLM generates answer -> return answer with citations where appropriate.

52. What happens when a document is uploaded into RAG?

The document is parsed, cleaned, chunked, converted into embeddings, enriched with metadata, and indexed in a vector/search store.

53. What happens when the user asks a question?

The query is embedded, relevant chunks are retrieved, those chunks are inserted into the prompt, and the LLM generates an answer based on the supplied context.

54. Does the vector DB store the original text?

Often yes. Systems commonly store the vector together with text or a reference to the original text plus metadata such as document ID, page, source and permissions.

55. Does RAG train the LLM?

No. Standard RAG retrieves information at runtime and places it in context. It does not modify the model weights.

56. Why not simply send the complete document to the LLM?

Large documents consume context, increase cost and latency, may exceed context limits, and can reduce signal-to-noise. Retrieval selects relevant information.

57. How do you choose chunk size?

It depends on document structure, embedding model, question types and retrieval strategy. Test multiple chunking configurations and evaluate retrieval quality rather than assume one universal size.

58. What happens if chunks are too small?

Context can become fragmented and important relationships may be lost.

59. What happens if chunks are too large?

Retrieval becomes less precise, irrelevant information enters the prompt, and token usage increases.

60. What is metadata filtering?

Restricting retrieval using attributes such as department, document type, user permissions, country, product or date before or alongside vector similarity.

61. What is hybrid search?

Hybrid search combines semantic/vector retrieval with lexical/keyword retrieval such as BM25.

62. Why use hybrid search?

Vector search is excellent for semantic similarity, while keyword search can perform better for exact identifiers, product codes, error messages, names and technical terminology.

63. What is reranking?

Reranking takes retrieved candidates and uses another scoring mechanism or model to reorder them according to relevance to the query.

64. Why rerank retrieved results?

Initial retrieval is optimized for efficiently finding candidates. A reranker can apply a more accurate but expensive relevance calculation to a smaller candidate set.

65. What is query rewriting?

Transforming the user's query into a clearer or retrieval-optimized query before searching the knowledge base.

66. What is query expansion?

Adding related concepts or generating alternative queries to improve retrieval coverage.

67. What is similarity threshold?

A threshold can reject retrieval results below an acceptable relevance score rather than always returning Top-K results.

68. Why isn't Top-K enough?

Top-K always returns K items even when all candidates are poor. Combining ranking with thresholds and other relevance checks can reduce irrelevant context.

69. What is prompt injection?

An attack where malicious instructions attempt to manipulate the model, override intended rules, reveal information, or cause unauthorized actions.

70. How do you protect a RAG application from prompt injection?

Separate instructions from retrieved data, treat retrieved content as untrusted, enforce authorization outside the model, restrict tools, validate inputs/outputs and never rely solely on prompts for security.

71. What is PII?

Personally identifiable information is data that can identify or relate to an individual, such as certain names, IDs, phone numbers or account information.

72. How would you protect sensitive data?

Use authentication, authorization, encryption, access-aware retrieval, data minimization, masking/redaction where needed, audit logging, secret management and appropriate retention policies.

73. What is RBAC in RAG?

Role-Based Access Control ensures users retrieve only documents they are authorized to access.

74. Why is RBAC particularly important in RAG?

Without permission-aware retrieval, an LLM could receive confidential chunks from documents the requesting user should never see.

75. What is conversation memory?

Information retained across turns so the application can maintain conversational context.

76. Is conversation memory the same as model training?

No. Memory provides context to the application/model. It does not normally change the model's trained weights.

77. What is short-term memory?

Information maintained for the current conversation or a limited interaction window.

78. What is long-term memory?

Persisted information that may be retrieved and used across sessions, subject to privacy and application requirements.

79. What is tool calling?

The model selects a predefined capability and produces arguments; the application executes it and returns the result to the model.

80. Give an example of tool calling.

A user asks 'Where is order 456?' The model chooses `get_order_status(orderId=456)`. The backend calls the order API and returns the result.

81. What is an agent loop?

Goal/input -> reason/decide -> select action -> execute tool -> observe result -> update state -> repeat or answer.

82. Agent vs chatbot?

A simple chatbot mainly generates conversational responses. An agent can dynamically decide to use tools and perform actions to achieve a goal.

83. What is agent state?

State contains information required across workflow steps, such as messages, tool results, intermediate decisions and task status.

84. Why use LangGraph instead of a simple agent loop?

It provides explicit state, nodes, transitions, branching, cycles and workflow control, useful for complex or production agent systems.

85. What is MCP?

Model Context Protocol is an open protocol for connecting AI applications to external tools and contextual data through standardized interfaces.

86. What is streaming?

Streaming returns generated content incrementally instead of waiting for the entire response to complete.

87. Why use streaming?

It improves perceived responsiveness, especially for long LLM outputs.

88. What is latency?

The time between submitting a request and receiving a useful response.

89. How can you reduce LLM cost?

Use appropriate models, reduce unnecessary context, improve retrieval, cache suitable results, limit output tokens, summarize history, batch where supported, and route simple requests to cheaper models.

90. What is model routing?

Selecting different models depending on task complexity, cost, latency, modality or risk.

91. What is caching in GenAI?

Reusing previously computed results such as embeddings, retrieval results, model responses, or provider-supported prompt/prefix caches where safe and appropriate.

92. What is semantic caching?

Reusing an existing response when a new query is semantically sufficiently similar, rather than requiring an exact string match.

93. What is an evaluation dataset?

A collection of representative questions/tasks and expected criteria used to measure application quality.

94. How do you evaluate a RAG system?

Evaluate retrieval separately from generation using metrics such as retrieval relevance/recall and answer correctness, faithfulness/groundedness, completeness and latency/cost.

95. What is faithfulness?

Whether the generated answer is supported by the retrieved or supplied context.

96. What is relevance?

Whether retrieved information or the generated answer actually addresses the user's query.

97. What is LLM-as-a-judge?

Using another LLM, guided by a rubric, to evaluate outputs for criteria such as relevance or correctness.

98. Can LLM-as-a-judge be wrong?

Yes. LLM judges can be biased or inconsistent, so important evaluations should combine clear rubrics, representative datasets, deterministic metrics where possible, and human review.

99. What is observability in GenAI?

Capturing traces and metrics across prompts, retrieval, tool calls, model calls, tokens, latency, errors, cost and quality.

100. How would you build an enterprise GenAI application?

Separate the solution into UI/API, orchestration, model gateway, RAG/search, tools, security, data stores, guardrails, evaluation and observability layers.

Part 3 - 30 Advanced Questions

101. What is ANN search?

Approximate Nearest Neighbor search efficiently finds vectors close to a query vector without exhaustively comparing against every vector.

102. Why approximate instead of exact search?

Exact nearest-neighbor search can become expensive at large scale. ANN trades a small amount of recall for significantly better performance.

103. What is HNSW?

Hierarchical Navigable Small World is a graph-based ANN indexing algorithm widely used for fast vector similarity search.

104. What is embedding dimensionality?

The number of numerical dimensions in an embedding vector. The exact dimensionality depends on the embedding model.

105. Can embeddings from different models be mixed?

Generally you should not directly compare vectors generated by different embedding models because their vector spaces are not guaranteed to be compatible. Re-embedding is normally required during migration.

106. Cosine similarity vs dot product?

Cosine focuses on vector direction after normalization considerations; dot product incorporates vector magnitudes. Which to use depends on how the embedding model was trained and provider recommendations.

107. What is retrieval recall?

It measures whether relevant documents are successfully included among retrieved candidates.

108. What is precision in retrieval?

It measures how much of what we retrieved is actually relevant.

109. What is MMR?

Maximum Marginal Relevance tries to balance relevance with diversity so retrieved chunks are not excessively redundant.

110. What is parent-child retrieval?

Small child chunks can be used for precise matching while larger parent sections are returned to provide richer context.

111. What is contextual retrieval?

It enriches chunks with additional document context so otherwise ambiguous chunks can be retrieved more accurately.

112. What is Graph RAG?

Graph-based RAG uses relationships/entities represented in a graph to support retrieval and reasoning over connected information, often alongside text/vector retrieval.

113. When would you prefer Graph RAG?

When questions depend heavily on relationships, multi-hop connections or entity networks rather than isolated text similarity.

114. How do you prevent context-window overflow?

Retrieve fewer/better chunks, rerank, compress context, summarize conversation history, limit tool output and budget tokens explicitly.

115. What is context compression?

Reducing retrieved content to the portions most relevant to the query before sending it to the model.

116. What is multi-query retrieval?

Generate multiple search formulations of the same information need, retrieve candidates for each, then combine and rank them.

117. How do you handle follow-up questions in RAG?

Maintain conversation state and, when necessary, rewrite the follow-up into a standalone retrieval query using relevant conversation context.

118. Why shouldn't the LLM directly enforce authorization?

LLM output is probabilistic and susceptible to manipulation. Authorization should be enforced deterministically in the application/data layer.

119. What is an agentic RAG system?

Instead of always following one fixed retrieval path, an agent decides whether and how to search, which source/tool to use, whether additional retrieval is necessary, and when enough evidence exists to answer.

120. RAG vs agentic RAG?

Traditional RAG generally follows a predefined retrieval pipeline. Agentic RAG lets the model dynamically choose retrieval strategies and tools.

121. What is multi-agent architecture?

Multiple specialized agents cooperate, such as a researcher, planner, developer and reviewer, coordinated through shared state or an orchestrator.

122. What is a supervisor agent?

An agent that coordinates specialized agents, decides which should handle tasks, and combines or routes their results.

123. What problems can multi-agent systems create?

Higher token usage, latency, coordination complexity, loops, conflicting decisions, larger attack surface and harder debugging/evaluation.

124. How do you prevent infinite agent loops?

Use maximum iteration limits, time/token/cost budgets, explicit termination conditions, state-machine constraints and failure handling.

125. What is human-in-the-loop?

Requiring human approval or review at selected points, particularly for sensitive, expensive, irreversible or high-impact actions.

126. How would you design a production RAG system?

Ingestion pipeline -> parsing/chunking -> embedding/indexing -> permission-aware retrieval -> hybrid search -> reranking -> context construction -> LLM -> output validation/citations -> evaluation, tracing and monitoring.

127. How would you debug a wrong RAG answer?

Determine whether failure occurred in query understanding, retrieval, reranking/context construction, or generation. Inspect retrieved chunks first before changing the LLM.

128. What if the correct document wasn't retrieved?

Investigate chunking, embedding quality, metadata filters, query rewriting, hybrid retrieval, Top-K, thresholds and indexing.

129. What if the correct document was retrieved but the answer was wrong?

Then retrieval may be working and the problem may be prompt construction, excessive/noisy context, conflicting evidence, model capability or insufficient grounding constraints.

130. How do you measure whether a GenAI application is production-ready?

Measure quality and groundedness alongside retrieval metrics, safety/security, latency, availability, cost, tool success rates, observability and real-user feedback.

The RAG Flow You Should Be Able to Explain

Offline ingestion: Documents -> parsing/cleaning -> chunking -> embedding model -> embeddings + metadata -> vector/search store.

Online query: User question -> query embedding -> vector/hybrid search -> Top-K candidates -> metadata filters/reranker -> relevant context -> prompt + question -> LLM -> grounded answer + sources.

Key interview line: The vector database does not generate the answer. It retrieves relevant information. The LLM uses that retrieved information as context to generate the response.

RAG vs Fine-tuning vs Prompt Engineering

Requirement	Technique
Tell model what to do	Prompt Engineering
Give model current/private knowledge	RAG
Modify behavior/style/task specialization	Fine-tuning
Find semantically related information	Embeddings + Vector Search
Perform an external action	Tool / Function Calling
Dynamically decide actions	Agent
Preserve conversation/application state	Memory / State
Restrict behavior/actions	Guardrails + deterministic controls
Determine whether output is good	Evaluation
Understand production behavior	Observability

Practical Scenarios - How to Describe Your Hands-on Experience

Scenario 1: Enterprise Document Q&A; Using RAG

I built a proof-of-concept enterprise document assistant using RAG. During ingestion, I extracted document text, split it into chunks, generated embeddings and stored them with metadata in a vector store. At query time, I embedded the question, retrieved relevant chunks and supplied them as grounded context to the LLM. This allowed the application to answer from private/domain-specific documents without retraining the model.

Scenario 2: Token Optimization

Tokens affected cost, latency and context usage. Instead of sending entire documents, I used RAG to retrieve only relevant chunks, controlled chunk count/size, limited output length and summarized unnecessary conversation history.

Scenario 3: Vector Database

I used vector search because the requirement was semantic retrieval. A user may ask 'How can employees claim travel expenses?' while the document says 'Employee airfare reimbursement procedure.' Embeddings help retrieve content based on meaning, while structured databases remain useful for metadata and transactional data.

Scenario 4: Chunking

I found that overly large chunks reduced retrieval precision while overly small chunks fragmented context. I experimented with chunk size and overlap and preserved metadata such as document, page and section.

Scenario 5: Hallucination Reduction

I grounded answers using retrieved trusted context, instructed the model to acknowledge insufficient evidence, and retained source metadata for citations. I would also measure faithfulness because RAG reduces hallucination but does not guarantee elimination.

Scenario 6: Semantic Search

I generated embeddings for both document chunks and the query, then used vector similarity to retrieve semantically related content even when wording differed.

Scenario 7: Hybrid Search

For exact identifiers such as error codes and product IDs, I combined keyword/BM25-style retrieval with semantic vector search. This gives strong exact-match behavior plus semantic coverage.

Scenario 8: Reranking

I retrieved a larger candidate set and reranked it with a stronger relevance model, then sent only the best few chunks to the LLM. This improves context quality without unnecessarily increasing prompt size.

Scenario 9: Caching and Cost

I measured token and latency hotspots, cached embeddings for unchanged documents, reused suitable results, reduced unnecessary context and used model routing so simpler requests could use cheaper/faster models.

Scenario 10: AI Agent and Tools

For an HR assistant, RAG handled policy documents while a live HR API returned the employee's current leave balance. I distinguish knowledge retrieval from live transactional data and actions.

Scenario 11: Agentic AI

For an IT support assistant, an agent could decide whether to search documentation, call a monitoring API, inspect ticket history or create a support ticket. For deterministic business processes, I prefer explicit workflows rather than unnecessary autonomy.

Scenario 12: Security and RBAC

Authentication identifies the user, while authorization determines what they can retrieve. I enforce document permissions at the retrieval/data layer so unauthorized chunks never reach the LLM.

Scenario 13: Prompt Injection

I treat retrieved content as untrusted data rather than instructions, separate system instructions from context, restrict tool permissions and enforce authorization outside the model. Prompt engineering alone is not a security boundary.

Scenario 14: Query Rewriting

For a follow-up such as 'How much does it cost?' after discussing Azure Container Apps, I rewrite the retrieval query into a standalone form such as 'What is the pricing model for Azure Container Apps?' before search.

Scenario 15: Production Architecture

I separate the application into UI/API, authentication and RBAC, orchestration, RAG/search, tools/APIs, model layer, guardrails, observability and evaluation. This makes security, debugging and scaling easier.

7-Day Interview Preparation Plan

Day	Focus
Day 1	GenAI, LLMs, Transformers, Tokens, Prompting, Temperature
Day 2	Embeddings, Vectors, Similarity, Vector Databases
Day 3	RAG, Chunking, Retrieval, Top-K, Metadata
Day 4	Hybrid Search, Reranking, Query Rewriting, RAG Evaluation
Day 5	Function Calling, Tools, Agents, Agentic RAG, LangChain/LangGraph
Day 6	Security, Prompt Injection, Guardrails, Evaluation, Observability
Day 7	Architecture, scenario questions and mock interview practice

Highest-priority chain to master: Tokens -> Embeddings -> Vector DB -> Semantic Search -> RAG -> Chunking -> Retrieval -> Reranking -> Prompt -> LLM -> Hallucination -> Tools -> Agents -> Evaluation.